Array Algorithms by Lemma Lookup: Worked Examples with Trace Equation Systems

Gidon Ernst
LMU Munich, Germany
gidon.ernst@gmail.com

October 2026

Abstract

A loop can be verified without an invariant if its effect is a known shape: a counter, a sum, a sequence of stores, a bisection of an interval. These notes show what it takes to verify array programs this way. Loops are described by equations over the traces of their variables; a database of lemmas states what loops of known shapes compute; and an engine rewrites the goal with the lemmas until it is proved. On worked examples, from a linear search to binary search, nested loops and the C programs of the MonoCera benchmarks, we show which lemmas each proof needs, which capabilities of the engine it relies on, and, for the programs that are not proved, what is missing. The examples are part of the verifier Cuvée (`examples/dafny`, `benchmarks/monocera`), and every result reported here was rerun.

1 Introduction

Proving a loop correct usually means finding an invariant. Many loops in practice need none: they count, sum, copy, fill, or search, and what they compute after n iterations has a closed form that a lemma can state once. Verification by lookup exploits this. Symbolic execution describes each loop by equations over the sequences of values its variables take, the *traces*; a database of lemmas rewrites traces of known shapes into their closed forms; and the postcondition, once the traces are gone, is a formula that the engine decides. Nothing is inferred. A loop whose shape has a lemma is proved within seconds; a loop without one stays open, and the goal that remains says which lemma is missing.

Three things must work together for this, and the examples of this document test each of them:

- **the representation** must expose the shapes: a counter must look like a counter, also when its variable is read by others, and an inner loop must be a term to which the lemmas of single loops apply (Section 2);
- **the engine** must find the shapes in the goal although the goal names traces by variables, splits facts over many hypotheses, and states arithmetic in its own normal form (Section 3);
- **the database** must contain the shapes, and its lemmas must be true for every term they match, not only for the loops they were written for (Sections 3.10 and 6).

The worked examples come in order of what they need. The linear search (Section 4) needs an iteration constraint named and instantiated at a position. Nested loops (Section 5) need nothing beyond the lemmas of single loops, because an inner loop is a value of the outer update; the transpose needs one lemma for the nest. Binary search (Section 6) needs a lemma over the whole loop, whose two variables are meaningless apart, and shows why such a lemma must be total to be sound. The C programs of the MonoCera benchmarks (Section 8) need the database alone. Selection sort, bubble sort, gnome sort, the merge and the distinctness check (Section 7) need a summary of what their loops compute and an induction or an invariant of the whole cycle, stated for the closed form. The programs that are not proved are discussed with what each needs (Section 9). Section 10 collects the principles of writing lemmas for lookup, and Section 11 the insights into the representation, the engine and soundness.

Notation. Programs and lemmas are in the Dafny dialect of Cuvée, where arrays are total maps $\mathsf{array}\langle\mathsf{int},\mathsf{int}\rangle$ and $a[i := v]$ is a store. We write $\lfloor X \rfloor$ for the history of a trace X, the list of its values before the last, $\mathsf{current}(X)$ for its last value, and $\mathsf{counter}(i, c, n)$ for the counter $\langle i, i + c, \ldots, i + nc\rangle$ of n steps. $\mathsf{swap}(x, p, q)$ abbreviates the double store $x[p := x[q]][q := x[p]]$. Capital letters K, I, S denote traces, and hatted letters $\hat{N}$ choices, lists of an outer loop that nothing defines, such as the numbers of iterations of an inner loop. Each example is run by the command line in the first line of its file, `-teq` to generate the trace equation system and `-prove` to prove it, with the modules of the database that the file names.

2 Trace Equation Systems

A loop is described by equations over *traces*, the sequences of values that its variables take, one value per iteration. This section defines the combinators that the equations use, how a loop is turned into a system of equations, and what such a system looks like for single loops, for loops whose variables read each other, and for nested loops.

2.1 Combinators

A trace of n steps has $n + 1$ values. Its last value is $\mathsf{current}(X)$, and the n values before it are the list $\lfloor X \rfloor$, the *history* of X; the j-th earlier value is $\mathsf{earlier}(j, \lfloor X \rfloor)$. The *counter* $\mathsf{counter}(i, c, n)$ is the trace $\langle i, i + c, \ldots, i + nc\rangle$ of n steps. Traces are built and observed by three combinators, each iterating over one or more lists of equal length, the *dependencies*, whose elements are bound pointwise:

- $\mathsf{trace}(x_0,\ \mathsf{for}\ D_1, \ldots, D_m\ \mathsf{do}\ x, y_1, \ldots, y_m \mapsto u)$, the trace from x_0 that steps to u with the next values y_i of the D_i;
- $\mathsf{maps}(\mathsf{for}\ D_1, \ldots, D_m\ \mathsf{yield}\ y_1, \ldots, y_m \mapsto e)$, the list of the values of e;
- $\mathsf{invs}(\mathsf{for}\ D_1, \ldots, D_m\ \mathsf{check}\ y_1, \ldots, y_m \mapsto \varphi)$, φ for all positions.

In Dafny the three are written `trace(x0, for D do x, y :: u)`, `maps(for D yield y :: e)` and `invs(for D check y :: φ)`, the history of a trace $\lfloor$`X`$\rfloor$ and its last value `current(X)`. The number of steps of a trace is never a term of the goal: it is the length of the counter of the iterations, $K = \mathsf{counter}(0, 1, n)$, and lemmas iterate over counters whose number of steps n they name directly. The type Nat of such numbers is a datatype with its embedding $\mathsf{integer}(n)$ into the integers and the inverse $\mathsf{natural}(i)$, which is 0 for negative i.

An update may consist of guarded cases, $\mathsf{case}\ \varphi_1 \Rightarrow e_1\ \ \mathsf{case}\ \varphi_2 \Rightarrow e_2$, which take the result of the case whose guard holds. Where no guard holds the update is unspecified, one fixed but unknown value per type. The guards of the cases that symbolic execution produces are complete for the iterations that the loop actually performs, which the iteration constraint (below) states, and disjoint, since they are the conditions of the branches of the loop body.

A trace may have several accumulators, for variables that read each other:

$$S = \mathsf{trace}(x_0, y_0,\ \mathsf{for}\ D\ \mathsf{do}\ x, y, d \mapsto u, v)$$

steps both components at once. Its components are read by $\mathsf{current}(S).0$, $\mathsf{current}(S).1$ and their histories by $\lfloor S \rfloor.0$, $\lfloor S \rfloor.1$, each an ordinary list that can be a dependency of further combinators. No term of a tuple type is ever built: a component is only read through these two observers.

2.2 From a loop to a system

Symbolic execution of a loop introduces the counter $K = \mathsf{counter}(0, 1, n)$ of a fresh number n of iterations and expresses the values of the modified variables after the loop as current values of traces. It records three kinds of facts:

- the *trace equations*, $X = \mathsf{trace}(x_0,\ \mathsf{for}\ \lfloor K \rfloor, \ldots\ \mathsf{do}\ x, k, \ldots \mapsto u)$, one per trace, whose update u is the effect of one execution of the body;

- the *iteration constraint*, $\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \lfloor X \rfloor, \ldots\ \mathsf{check}\ k, x, \ldots \mapsto \mathit{test})$: the loop test held at the start of every iteration;
- the *exit*, $\neg \mathit{test}$ for the current values: the loop has stopped.

Each variable reads some others in the update. The modified variables are partitioned by the strongly connected components of this read relation, ordered such that each comes after the components it reads, and each component becomes *one* trace: a variable in no cycle gets a trace of one accumulator, a cycle gets a trace of several. Each trace has the counter, the histories of the earlier components it reads, and the choices of the body among its dependencies. Since the components form an acyclic graph, no trace refers to a later one, and the system can be read from top to bottom, each equation defining a trace in terms of the previous ones.

A single loop. The linear search `i := 0; while i < n && a[i] != x { i := i + 1; }` gives

$$K = \mathsf{counter}(0, 1, n') \qquad I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ i, k \mapsto i + 1) \qquad i = \mathsf{current}(I)$$
$$\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, i \mapsto i < n \wedge a[i] \neq x) \qquad \neg(i < n \wedge a[i] \neq x).$$

The trace of i is a counter, the constraint states the test for the earlier values of i, the exit for the current one.

A cycle. In the binary search, `lo` and `hi` read each other through the probe `(lo + hi) / 2`, and the two variables form one trace of two accumulators:

$$S = \mathsf{trace}(0, n,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ lo, hi, k \mapsto \mathsf{case}\ G \Rightarrow (hi + lo)/2 + 1\ \ \mathsf{case}\ \neg G \Rightarrow lo,\ \ \mathsf{case}\ G \Rightarrow hi\ \ \mathsf{case}\ \neg G \Rightarrow (hi + lo)/2)$$

with G for $a[(hi+lo)/2] < x$, the exit values $lo = \mathsf{current}(S).0$, $hi = \mathsf{current}(S).1$, and the constraint $\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \lfloor S \rfloor.0, \lfloor S \rfloor.1\ \mathsf{check}\ k, lo, hi \mapsto lo < hi)$. Equations of one accumulator each would be recursive through each other; one equation over the pair is not. The per-variable view of a component is still available when a lemma needs it, as a *projection* (Section 3.4).

Nested loops. An inner loop is summarized while the body of the outer loop is executed, as a system of its own over a fresh number of inner iterations. This system is *closed*: its counter, its traces and its exit values are substituted by their definitions, so that only the number of inner iterations remains fresh, and the inner loop is a value of the outer update. In the nested sum

```
c := 0; i := 0;
while i < m { j := 0; while j < n { j := j + 1; } c := c + j; i := i + 1; }
```

the final value of j in an outer iteration is $\mathsf{current}(\mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n_7) \rfloor\ \mathsf{do}\ j, k \mapsto j + 1))$ for the number n_7 of inner iterations of that outer iteration, and the trace of c is

$$C = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor, \hat{N}\ \mathsf{do}\ c, k, n_7 \mapsto c + \mathsf{current}(\mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n_7) \rfloor\ \mathsf{do}\ j, k' \mapsto j + 1))).$$

The numbers of inner iterations form a list $\hat{N}$ of the outer loop, a *choice*: a dependency that nothing defines, about which the constraint of the outer loop states what the inner loop's own constraint and exit say,

$$\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \ldots, \hat{N}\ \mathsf{check}\ k, \ldots, n_7 \mapsto i < m \wedge \mathsf{invs}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n_7) \rfloor, \ldots\ \mathsf{check}\ \ldots \mapsto j < n) \wedge \neg(\mathsf{current}(\ldots) < n)).$$

Lemmas on single loops apply to the inner loop as it stands in the update, since it is a term like any other; once the constraint determines the numbers of iterations, e.g. $n_7 = \mathsf{natural}(n)$ in every outer iteration, the choice becomes the constant list $\lfloor \mathsf{consts}(n', \mathsf{natural}(n)) \rfloor$ and the inner traces have a known number of steps.

The goal. The goal of a method is that its postcondition follows from the facts of the path through it, with the traces as variables defined by their equations: $K = \mathsf{counter}(0, 1, n')$, $S = \mathsf{trace}(\ldots)$, the exit values $lo = \mathsf{current}(S).0$, the iteration constraint and the exit. The equations of the outermost loops are kept as equations, which the engine follows where a lemma needs the definition of a trace variable; inner loops are closed, as above.

3 The Engine

The goals of Section 2 are proved by rewriting with a database of lemmas: an equation $l = r$ or an equivalence $l \Leftrightarrow r$ is applied from left to right wherever its left-hand side matches, provided its side conditions hold. A goal is proved when it rewrites to true. Nothing is inferred: the lemmas state what loops of a certain shape compute, and the engine's job is to find the loops of the goal among those shapes. That job is harder than it looks, and the engine is designed around what makes it hard. Each design decision below answers a difficulty that the examples of this document met; the insight is stated first, then the mechanism.

3.1 The facts form one structure, not a list

Insight: a lemma must see a term in whatever form the goal states it. The goal names traces by variables, $K = \mathsf{counter}(0, 1, n)$, $I = \mathsf{counter}(0, 1, n)$, $b = \mathsf{current}(B)$, and a lemma about counters must match $\lfloor K \rfloor$ as well as $\lfloor \mathsf{counter}(0, 1, n) \rfloor$, and $\mathsf{sorted}(b, 0, n)$ as well as $\mathsf{sorted}(\mathsf{current}(B), 0, n)$.

The facts of the context are kept in a congruence closure: a union-find over their terms, where equal arguments make applications equal. The *definitions* of a variable are the members of its class that are not variables, in both orientations of every equation and transitively, so $K = I$ follows from two equations with the same counter, and $x = f(y)$ with $y = z$ gives $f(z)$. When a pattern does not match a variable, the matcher tries its definitions. Only variables have definitions: a term matched through a member of its class that contains the term would be rewritten forever.

3.2 Closed conditions are looked up, not proved again

Insight: a side condition whose variables are all bound is a question about the facts, not a new goal. A lemma's side conditions are matched against the facts while they have open variables, which binds them. Once a condition is closed, it is first looked up: it holds if it is congruent to a fact modulo the classes, and a condition with binders, such as an iteration constraint $\mathsf{invs}(\mathsf{for}\ \ldots\ \mathsf{check}\ \ldots)$, holds if its canonical form equals that of a fact. The canonical form renames bound variables by position and sorts dependencies, cases, conjuncts and the sides of equations, so that alpha-variants and reorderings coincide. Only a condition that is not found is decided by simplification. Without the lookup, a summary lemma that needs the iteration constraint of an inner loop as a condition must restate it with fresh variables and identify them with the ones of the goal by equations; with it, the condition is written as it is meant. The instance is simplified before it is looked up, so a fact whose dependencies include one that its body does not use is not found by a condition without it (Section 7.1).

3.3 Arithmetic is decided, not rewritten

Insight: the bounds that make array reads meaningful are linear facts, and a decision procedure should supply them. The context decides linear arithmetic over the integers by a simplex with bounds; strict comparisons are offset by one, and an atom t/k for a positive constant k brings its bounds $k \cdot (t/k) \le t \le k \cdot (t/k) + k - 1$. So the midpoint of a binary search lies in its interval, $lo \le (hi + lo)/2 < hi$ for $lo < hi$, without a lemma. Solutions of linear equations are substituted into polynomials.

3.4 Views: per-variable lemmas apply to cycles

Insight: the database is written for single accumulators, but loops have cycles; the engine should translate, not the database. Most lemmas describe a trace of one accumulator over a counter, a sum, a store, a bound. A variable of a cycle is a component of a trace of several accumulators, $\mathsf{current}(S).0$. The engine treats such a read as a marker and matches it modulo its *view*: the trace of that accumulator alone, with the other accumulators bound over their histories $\lfloor S \rfloor.j$ as further dependencies. A lemma about $\mathsf{current}(\mathsf{trace}(x_0,\ \mathsf{for}\ \lfloor K \rfloor, hs\ \mathsf{do}\ x, k, h \mapsto \ldots))$ then applies to $\mathsf{current}(S).0$ with $hs := \lfloor S \rfloor.1$. The histories are inert as dependencies, so the projections do not unfold each other, and a component is not projected again inside its own view. A marker is replaced by its view once the view reads no component of the trace, e.g. a counter by its value. Views are memoized by the facts that mention their variables, so that positions with the same relevant facts share them.

3.5 Whole cycles: a lemma may name the loop

Insight: some loops are only meaningful as a whole, and the iteration constraint belongs inside the update. A pattern with several accumulators matches a trace of as many, component by component: the initial values pairwise, the results in the context of the update. In that context the histories of the trace denote the accumulators, so every iteration constraint over the histories holds for the accumulators: inside the update of the binary search, $lo < hi$ is a fact. This is what makes a lemma like

$$\begin{aligned}&\mathsf{trace}(l_0, h_0,\ \mathsf{for}\ ks\ \mathsf{do}\ lo, hi, k \mapsto \mathsf{case}\ G(M(lo,hi)) \wedge lo < hi \Rightarrow M(lo,hi) + 1\ \ldots)\\ &\quad = \mathsf{bisection}(l_0, h_0, ks, M, G, H)\end{aligned}$$

match a program whose guards do not mention $lo < hi$: the conjunct follows from the guard of the program in the context of its update. The same mechanism decides guards that depend on the constraint in every rewrite inside an update, not only for whole cycles.

Higher-order variables of a pattern, the probe M and the guards G, H, are bound by abstracting the matched term over the bound variables, as in the higher-order pattern fragment, and an instance applied to arguments is beta-reduced.

3.6 Case lists are matched in one direction

Insight: a lemma may be total where the program is partial, as long as the context excludes the difference. An update of cases is matched by requiring every case of the program to be covered by a case of the pattern, whose result matches and whose guard follows from the program's guard. Cases of the pattern that cover none are ignored. This is an equation wherever the program has a case, and the guards of the program are complete in the context of its update by construction: the iteration constraint states that the loop test held, and the cases are the branches of the body. So a lemma may add a last case for the complement of its guards, e.g. $\mathsf{case}\ \neg(lo < hi) \vee \neg(G \vee H) \Rightarrow lo$, which keeps the state. The program has no such case, the context excludes it, and the lemma becomes true for *every* update, not only those of a loop that stops in time. Section 6 shows why this matters for soundness.

3.7 Forward facts: triggers instead of rewrites on atoms

Insight: many facts about a loop are needed where only its term occurs. A bound of the current value of a monotone trace, or the bounds of a binary search's result, is not a rewrite of anything: no atom of the goal mentions it. A lemma marked `{:fw}`, `requires C ensures` φ, is assumed wherever a term matching its *trigger* occurs in a fact; the trigger is the term of φ with the most variables, below its boolean and arithmetic structure. The trigger is matched like a left-hand side, through definitions and views, and the instance names the term as it occurs, so that it reaches the classes and the linear constraints of the goal: $\mathsf{current}(\mathsf{bisection}(\ldots)).0 \leq \mathsf{cut}(G, l_0, h_0)$ is assumed as

$\mathsf{current}(S).0 \leq \mathsf{cut}(G, 0, n)$, and the cut is then rewritten by the database to a partition point of the array. Triggers are retried while instances are added, since one instance may enable another; this terminates, as the terms are fixed.

3.8 Suspension: a rule does not prove its own conditions

Insight: a condition that mentions the rule's own left-hand side must not be rewritten by the rule. While the side conditions of a rule are evaluated, the rule is suspended, and while the view of a component is computed, that component is not projected. Without the suspension, a lemma about $\mathsf{current}(L)$ with the exit $\mathsf{current}(H) \leq \mathsf{current}(L)$ as a condition rewrites the condition by itself and never terminates; with it, the condition may mention the left-hand side.

3.9 What remains a goal

A goal that remains is split by cases on its first disjunctive premise, up to four deep, each case in a fresh context. A goal without traces and other functions than those of arithmetic, over at most four variables, is refuted if it evaluates to false at an assignment of small values; it becomes false, and the program is reported incorrect. All other goals are printed as they remain, with the lemmas that were applied, so that a residual goal points to the missing lemma.

3.10 The lemma database

The database is a set of Dafny files: an entry point, injected with every system, with the declarations of the combinators, counters, the functions on lists and the lemmas on traces; and optional modules for aggregations (sums, counts, minima and maxima), arithmetic, arrays (stores and their reads), counters, searching (no occurrence in a prefix, the bisection of an interval), sorted arrays (sortedness, permutations, partition points), strided stores and counters modulo a bound. A lemma is a rewrite rule (`{:rw}`), a forward fact (`{:fw}`), or the definition of a function that is folded where its body occurs (`{:fold}`). The lemmas of the entry point and of the modules are proved in Lean 4 against statements generated from the files, with the trace combinators defined by recursion and guarded cases as conditionals that end in unspecified. Where a lemma has arbitrary guards, it ends in a case for their complement (Section 3.6), so that complete guards are not a premise. What the proofs still assume is listed with each lemma: reads of the lists of a loop at its iterations, prefixes not longer than the list, lists of equal length. The lemmas written for a single program, at the beginning of its file, are not checked; Section 10.3 discusses how to keep them sound.

4 Linear Search: a Constraint Named and Instantiated

4.1 The program

```
method lsearch(a: array<int, int>, n: int, x: int) returns (i: int)
  requires 0 <= n
  ensures 0 <= i <= n
  ensures i < n ==> a[i] == x
  ensures forall k: int :: 0 <= k < i ==> a[k] != x
{
  i := 0;
  while i < n && a[i] != x { i := i + 1; }
}
```

The first index of x among $a[0..n)$, or n. The test reads the array, so the number of iterations is not determined by n.

4.2 The system

$$K = \mathsf{counter}(0, 1, n') \qquad I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K \rfloor\ \mathsf{do}\ i, k \mapsto i + 1) \qquad i = \mathsf{current}(I) \tag{1}$$

$$\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \lfloor I \rfloor\ \mathsf{check}\ k, i \mapsto i < n \wedge a[i] \neq x) \tag{2}$$

$$\neg(i < n \wedge a[i] \neq x) \tag{3}$$

The run is `-teq -split -prove`: the postcondition is split into one goal per conjunct, the quantified one as $0 \leq k < i \wedge a[k] = x \Rightarrow \mathsf{false}$.

4.3 What the proof needs

From the database, the counter. I is a counter by counter_def, so $i = \mathsf{integer}(n')$ by current_counter. Inside the constraint the value of i in iteration k is k (earlier_counter), and the constraint splits into $\mathsf{invs}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n') \rfloor\ \mathsf{check}\ k \mapsto k < n)$, which invs_counter_upper_nonnegative turns into $\mathsf{integer}(n') \leq n$ for $0 \leq n$, and the search constraint $\mathsf{invs}(\mathsf{for}\ \lfloor \mathsf{counter}(0, 1, n') \rfloor\ \mathsf{check}\ k \mapsto a[k] \neq x)$. The first two postconditions follow, and the third from the exit (3).

For the last postcondition, the constraint at a position. The goal needs $a[k] \neq x$ for a k below i, which is the search constraint at position k. A rule cannot take the constraint itself as a side condition and instantiate it: matching an iteration constraint as a condition builds the context of its combinator, in which the reads it contains are rewritten, and a rule general enough to match there recurs (Section 10.4). Instead, one lemma names what the constraint says, and a second one uses the name:

not_occurs_selects_def :

$$\mathsf{invs}(\mathsf{for}\ \lfloor \mathsf{counter}(i, 1, n) \rfloor\ \mathsf{check}\ k \mapsto a[k] \neq x) \iff \neg\mathsf{occurs}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(i, 1, n) \rfloor))$$

select_not_occurs_selects :

$$\neg\mathsf{occurs}(x, \mathsf{selects}(a, \lfloor \mathsf{counter}(i, 1, n) \rfloor)) \ \wedge\ i \leq k < i + \mathsf{integer}(n) \implies (a[k] = x) \iff \mathsf{false}$$

The first replaces the constraint by the atom "x does not occur among the values of a at $0, \ldots, n'-1$". The second is a rule on the atom $a[k] = x$ of the goal: its first condition is matched against the atom just produced, which binds $i := 0$ and $n := n'$; its bounds are closed and decided by the arithmetic from $0 \leq k < i$ and $i = \mathsf{integer}(n')$. The hypothesis $a[k] = x$ becomes false.

The same pair is in the module for searching, there with the second lemma as a forward fact (Section 3.7): triggered by the read $a[k]$, it assumes $a[k] \neq x$ wherever the read occurs, rather than rewriting one particular atom. The pattern is general: an iteration constraint becomes a named fact about the prefix that the loop visited, and a second lemma uses the fact at any position in the prefix.

5 Nested Loops: the Inner Loop as a Value

Four nested loops close with the database alone: an inner counter (`nested_counter`), its final values summed (`nested_sum`), a variable incremented by the inner loop (`nested_mult`), and the sums of the rows of a matrix stored by the outer loop (`row_sums`). A fifth, the transpose, needs one lemma that states what the two loops together compute. This section shows what the closed form of nested loops of Section 2 provides and what it leaves to a lemma.

5.1 The nested sum

```
method main(m: int, n: int) returns (c: int)
  requires 0 <= m && 0 <= n
  ensures c == m * n
{
  var i: int; var j: int;
  c := 0; i := 0;
```

```
  while i < m {
    j := 0;
    while j < n { j := j + 1; }
    c := c + j;
    i := i + 1;
  }
}
```

With $J_7 = \mathsf{current}(\mathsf{trace}(0,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n_7)\rfloor\ \mathsf{do}\ j,k \mapsto j+1))$ for the final value of j in an outer iteration with n_7 inner iterations, the system of the outer loop is

$$
\begin{aligned}
&K = \mathsf{counter}(0,1,n') \qquad c = \mathsf{current}(C) \qquad i = \mathsf{current}(I)\\
&C = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K\rfloor, \hat{N}\ \mathsf{do}\ c,k,n_7 \mapsto \mathsf{case}\ \neg(J_7 < n) \Rightarrow c + J_7) \qquad (4)\\
&I = \mathsf{trace}(0,\ \mathsf{for}\ \lfloor K\rfloor, \hat{N}\ \mathsf{do}\ i,k,n_7 \mapsto \mathsf{case}\ \neg(J_7 < n) \Rightarrow i + 1)\\
&\quad \mathsf{invs}(\mathsf{for}\ \lfloor K\rfloor, \lfloor C\rfloor, \lfloor I\rfloor, \hat{N}\ \mathsf{check}\ k,c,i,n_7 \mapsto i < m\\
&\qquad\qquad \wedge\ \mathsf{invs}(\mathsf{for}\ \lfloor \mathsf{counter}(0,1,n_7)\rfloor, \lfloor \mathsf{trace}(0,\ \mathsf{for}\ \ldots\ \mathsf{do}\ \ldots)\rfloor\ \mathsf{check}\ k', j \mapsto j < n) \wedge \neg(J_7 < n)) \qquad (5)
\end{aligned}
$$

and the exit $\neg(i < m)$. The update has one case, guarded by the exit of the inner loop: the path through the body passes the inner loop only once it has stopped. The inner loop occurs as a term in the updates and in the constraint; only the numbers $\hat{N}$ of its iterations are fresh.

What the proof needs. Everything is a lemma on single loops, applied inside the outer combinators:

1. The inner trace is a counter (counter_def), so $J_7 = \mathsf{integer}(n_7)$ (current_counter). This happens inside the update and inside the constraint alike, since the inner loop is a term like any other.
2. Inside the constraint (5), the inner test over the counter becomes $\mathsf{integer}(n_7) \leq n$ (invs_counter_upper_nonnegative), and with the inner exit $\neg(\mathsf{integer}(n_7) < n)$ the number of inner iterations is determined: $\mathsf{integer}(n_7) = n$ in every outer iteration.
3. A choice whose values the constraint determines is a constant list (consts_choice): $\hat{N} = \lfloor \mathsf{consts}(n', \mathsf{natural}(n))\rfloor$. Its reads in the updates become $\mathsf{natural}(n)$ (earlier_consts), and the guard $\neg(J_7 < n)$ becomes true.
4. The trace of c adds n in every iteration: it is folded into sums over a constant list (sums), whose current value is $n \cdot \mathsf{integer}(n')$ (current_sums, sum_consts). The outer constraint and exit give $\mathsf{integer}(n') = m$, and the arithmetic substitutes the solution into the polynomial: $c = m \cdot n$.

`nested_counter` stops after the second step, and `nested_mult` differs in the third: the inner loop increments the outer variable itself, so its trace is a counter from the previous value r, whose final value is $r + \mathsf{integer}(n_7)$, and the outer trace of r adds n per iteration as above.

Why the inner loop must be a value. If the inner loop were described by fresh lists of the outer loop, one per inner trace and its final value, each with its own iteration constraint, then the facts about one of them (the inner counter is $\mathsf{counter}(0,1,n_7)$) would sit in a different combinator than the facts that need them (the final value is the current value of that counter), and every substitution between them would be a lemma of its own, one per kind of inner loop. As a value of the update, the inner loop is one term, and the lemmas on single loops apply to it where it occurs, with the facts of the outer iteration in the context. The only thing that cannot be a value is the number of inner iterations, which is what the inner exit determines: it stays a choice, and consts_choice resolves it as soon as the constraint fixes it.

5.2 The row sums

```
method main(A: array<int, array<int, int>>, m: int, n: int) returns (s: array<int, int>)
  requires 0 <= m && 0 <= n
  ensures forall i: int :: 0 <= i < m ==>
    s[i] == sum(selects(A[i], ⌊counter(0, 1, natural(n))⌋))
```

```
{
  var i: int; var j: int; var t: int;
  i := 0;
  while i < m {
    t := 0; j := 0;
    while j < n { t := t + A[i][j]; j := j + 1; }
    s := s[i := t];
    i := i + 1;
  }
}
```

The inner loop sums row i, the outer loop stores the sum at position i. The update of the trace of s is

$$s[i := \mathsf{current}(\mathsf{trace}(0,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n_{10})\rfloor, \lfloor\mathsf{trace}(0,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n_{10})\rfloor\ \mathsf{do}\ j,k \mapsto j+1)\rfloor \\ \mathsf{do}\ t,k,j \mapsto t + A[i][j]))],$$

with the inner trace of j among the dependencies of the inner trace of t. The numbers of inner iterations are resolved to $\mathsf{natural}(n)$ as in the nested sum. Then the database does the rest: the inner trace of j is a counter, the reads $A[i][j]$ over it are $\mathsf{selects}(A[i], \lfloor\mathsf{counter}(0,1,\mathsf{natural}(n))\rfloor)$ (selects), the inner trace of t is a sums over them, and its current value is their sum (current_sums). The outer trace stores these sums at the positions of a counter (stores), and the read $s[i]$ of the postcondition is the stored value at i, by current_stores with the membership and the position of i in the counter (occurs_counter_up, indexof_counter_up): exactly the sum that the postcondition states. Nothing in this proof is about nested loops; it is the lemmas of single loops, applied in the order in which the terms appear.

5.3 The transpose: one lemma for the nest

```
lemma {:rw} current_transpose(B0: array<int, array<int, int>>, A: array<int, array<int, int>>,
                              m: int, n: int, i: int, j: int)
  requires 0 <= i < m && 0 <= j < n
  ensures current(trace(B0, for ⌊counter(0, 1, natural(m))⌋ do B, k ::
      current(trace(B, for ⌊counter(0, 1, natural(n))⌋ do Bi, ki ::
        Bi[ki := Bi[ki][k := A[k][ki]]]))))[j][i]
    == A[i][j]

method main(A: array<int, array<int, int>>, m: int, n: int)
  returns (B: array<int, array<int, int>>)
  requires 0 <= m && 0 <= n
  ensures forall i: int, j: int :: 0 <= i < m && 0 <= j < n ==> B[j][i] == A[i][j]
{
  var i: int; var j: int;
  i := 0;
  while i < m {
    j := 0;
    while j < n { B := B[j := B[j][i := A[i][j]]]; j := j + 1; }
    i := i + 1;
  }
}
```

The inner loop stores column i of A into row positions of B, a store into an array that is itself an element of B. No lemma of the database describes two-dimensional stores, so the program states one: the left-hand side is the nest as the system has it once the numbers of inner iterations are resolved to $\mathsf{natural}(n)$, the outer trace whose update is the current value of the inner trace, and the right-hand side the element of the transpose. The lemma is a summary of the whole nest in one line, with the side conditions that make the read meaningful. The program is otherwise proved by the database, as the row sums: the counters, the choice, and the read of the postcondition at j, i within the bounds.

What it needs. The lemma matches because its left-hand side is the term of the goal: an outer trace over a counter whose update contains the inner trace as a value. A summary of the inner loop alone would have been a rule on $\mathsf{current}(\mathsf{trace}(B,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,\mathsf{natural}(n))\rfloor\ \mathsf{do}\ \ldots))$ with the column as a named function, followed by a rule on the outer trace of these columns; the closed form lets one lemma state the result of both, at the price of a lemma specific to the nest. The same file in the folder of the algorithms keeps two folds, of the columns and of the outer results, and four lemmas written for inner loops described by fresh lists; they do not match the closed form, and one goal remains (Section 9).

6 Binary Search: a Lemma over a Whole Cycle

Binary search is the standard example of a loop whose variables are only meaningful together: lo and hi narrow an interval through a probe computed from both. This section shows what it takes to prove it by lookup: a lemma that names the loop as a whole, two facts about the named loop, and the vocabulary of sorted arrays that turns the facts into the postcondition.

6.1 The programs

```
method search(a: array<int, int>, n: int, x: int) returns (i: int)
  requires 0 <= n && sorted(a, 0, n)
  ensures 0 <= i ==> i < n && a[i] == x
  ensures i < 0 ==> forall k: int :: 0 <= k < n ==> a[k] != x
{
  var lo: int; var hi: int; var mid: int;
  lo := 0; hi := n; i := -1;
  while lo < hi {
    mid := (lo + hi) / 2;
    if a[mid] < x { lo := mid + 1; }
    else if x < a[mid] { hi := mid; }
    else { i := mid; break; }
  }
}
```

The second program, the lower bound, has no third branch: `else { hi := mid; }`, and returns lo with the postconditions $0 \le lo \le n$, $a[k] < x$ for $k < lo$ and $x \le a[k]$ for $lo \le k < n$. Both take the array sorted on $[0, n)$ and nothing else.

6.2 The system

The two variables form one trace of two accumulators. With $M(lo, hi) = (hi + lo)/2$, $G(m) = a[m] < x$ and $H(m) = x < a[m]$,

$$S = \mathsf{trace}(0, n,\ \mathsf{for}\ \lfloor K\rfloor\ \mathsf{do}\ lo, hi, k \mapsto \mathsf{case}\ G(M) \Rightarrow M + 1\ \ \mathsf{case}\ H(M) \Rightarrow lo,\ \mathsf{case}\ G(M) \Rightarrow hi\ \ \mathsf{case}\ H(M) \Rightarrow M) \tag{6}$$

$$\mathsf{invs}(\mathsf{for}\ \lfloor K\rfloor, \lfloor S\rfloor.0, \lfloor S\rfloor.1\ \mathsf{check}\ k, lo, hi \mapsto lo < hi) \tag{7}$$

$$\mathsf{invs}(\mathsf{for}\ \lfloor K\rfloor, \lfloor S\rfloor.0, \lfloor S\rfloor.1\ \mathsf{check}\ k, lo, hi \mapsto G(M) \vee H(M)) \tag{8}$$

with $lo = \mathsf{current}(S).0$, $hi = \mathsf{current}(S).1$. The second constraint (8) is there because of the `break`: an iteration that finds x leaves the loop, so every iteration that completed took one of the two other branches. After the loop there are two paths: the normal exit with $hi \le lo$ and $i = -1$, and the `break` with $lo < hi$, $a[M(lo, hi)] = x$ and $i = M(lo, hi)$.

6.3 What the proof needs

A name for the loop. The database defines the bisection of an interval with an arbitrary probe and two guards, and folds it where its body occurs:

```
function {:teq} {:fold} bisection(l0: int, h0: int, ks: List<int>, M: (int, int) -> int,
                                  G: int -> bool, H: int -> bool): Trace<(int, int)>
{
  trace(l0, h0, for ks do lo, hi, k ::
        case G(M(lo, hi)) && lo < hi => M(lo, hi) + 1
        case H(M(lo, hi)) && !G(M(lo, hi)) && lo < hi => lo
        case !(lo < hi) || !(G(M(lo, hi)) || H(M(lo, hi))) => lo,
        case G(M(lo, hi)) && lo < hi => hi
        case H(M(lo, hi)) && !G(M(lo, hi)) && lo < hi => M(lo, hi)
        case !(lo < hi) || !(G(M(lo, hi)) || H(M(lo, hi))) => hi)
}
```

The fold matches the trace (6) as a whole (Section 3.5). The probe M is bound from the result $M(lo, hi) + 1$ by abstraction, G and H from the guards; the conjunct $lo < hi$ of the definition's guards follows from the first constraint (7), which holds inside the update, and $\neg G(M)$ in the second case follows from the program's guard $x < a[m]$. The third case covers no case of the program and is ignored (Section 3.6). The result is $S = \mathsf{bisection}(0, n, \lfloor K \rfloor, M, G, H)$.

Two facts about the named loop. If G holds on a prefix of $[l_0, h_0)$, the lower end stays within l_0 and the *cut* of G, the first index where it fails; if H holds on a suffix, the upper end stays within the cut of $\neg H$ and h_0:

```
lemma {:fw} {:teq} bisection_lo(l0: int, h0: int, ks: List<int>, M: (int, int) -> int,
                                G: int -> bool, H: int -> bool)
  requires l0 <= h0
  requires forall l: int, h: int :: l < h ==> l <= M(l, h) && M(l, h) < h
  requires forall p: int, q: int :: l0 <= p && p <= q && q < h0 && G(q) ==> G(p)
  ensures l0 <= current(bisection(l0, h0, ks, M, G, H)).0 <= cut(G, l0, h0)
```

and bisection_hi symmetrically, $\mathsf{cut}((j) \Rightarrow \neg H(j), l_0, h_0) \leq \mathsf{current}(\ldots).1 \leq h_0$ under the condition that H holds on a suffix. They are forward facts, triggered by $\mathsf{current}(S).0$ and $\mathsf{current}(S).1$ in the goal. Their conditions are decided after the instantiation: the probe lies strictly inside the interval by the bounds of division (Section 3.3), and the guards hold on a prefix, respectively a suffix, since the rules below turn the comparisons $a[q] < x$ for indices in range into comparisons with a partition point, $q < \mathsf{lower}(a, n, x)$, after which the condition is linear.

The vocabulary of sorted arrays. The cuts become partition points: $\mathsf{lower}(a, n, x)$, below which the elements are less than x, and $\mathsf{upper}(a, n, x)$, from which they are greater: cut_lower rewrites the cut of $a[j] < x$, and the cut of $\neg(x < a[j])$, i.e. of $a[j] < x + 1$, is $\mathsf{lower}(a, n, x + 1) = \mathsf{upper}(a, n, x)$ (lower_succ). For an index in range, the comparisons with x are comparisons with the partition points (lower, upper, $a[m] = x \Leftrightarrow \mathsf{lower} \leq m < \mathsf{upper}$ by equal), and the partition points lie within $0 \leq \mathsf{lower} \leq \mathsf{upper} \leq n$ (the forward fact bounds).

The postconditions. At the normal exit, $\mathsf{upper} \leq hi \leq lo \leq \mathsf{lower}$, so the range $[\mathsf{lower}, \mathsf{upper})$ of the elements equal to x is empty and no $a[k]$ equals x. At the `break`, $a[M] = x$ and $M < hi \leq n$. For the lower bound, H is $x \leq a[m]$, the negation of G; the cut of $\neg H$ is the cut of G, so $lo \leq \mathsf{lower} \leq hi$, and at the exit $hi \leq lo$ gives $lo = \mathsf{lower}(a, n, x)$, from which the two quantified postconditions follow by lower and lower_le. Both programs close in under a minute.

6.4 Why the definition has three cases

The program's update has two cases. A definition with only those two, guarded by $lo < hi$, would be *unspecified* in an iteration where $lo < hi$ fails or neither guard holds, and bisection_lo, stated for any number of steps, would bound that unspecified value. Take $l_0 = h_0$, one step, and G, H false: all conditions of bisection_lo hold, the cut of G on the empty interval is l_0, and the lemma says that the first component of unspecified equals l_0, for every l_0, which derives false. The facts are true of the loops that the programs run, whose iterations all satisfy the constraints, but not of every trace that the definition denotes.

The third case removes the gap: where the program's step is undefined, the definition keeps the state, which keeps the bounds. The bounds now hold for every run of the definition, and they are proved by induction over the steps, carrying $l_0 \le lo \le \mathsf{cut}(G)$ and $hi \le h_0$ (for hi: $l_0 \le lo$ and $\mathsf{cut}(\neg H) \le hi \le h_0$). The second case excludes G, so that the guards are disjoint: where G and H both hold the first case is taken, which the bound also allows, since $G(m)$ and G holding on a prefix give $m < \mathsf{cut}(G)$. The fold still matches the program because the extra case is excluded by the context of the update.

Why H cannot be replaced by $\neg G$. In the context of the update, the program's guards are disjoint and complete, so $H(m)$ and $\neg G(m)$ agree at every probe that the loop visits. A definition with G alone, moving hi down wherever G fails, matches the programs as well, but its fact about hi can only be $\mathsf{cut}(G) \le hi$: a run of that definition may move hi onto a position that holds x, which the search leaves by the `break`. The search needs $\mathsf{upper} \le hi$, i.e. the cut of $\neg H$, a property of H on the whole interval rather than at the probes. The definition must therefore encode the exit itself: a probe at which neither guard holds keeps the state, and hi moves only to positions where H holds.

7 Sorts, Cycles and Flags

Five algorithms with nested loops or cycles are proved by lemmas written for the closed form of Section 2: bubble sort and selection sort, whose inner loops have a known length; gnome sort and the merge, whose variables form cycles; and the distinctness check, whose inner loop is a flag. Each file states, at its beginning, a summary of what its loops compute and the facts that turn it into the postcondition. Each proof closes within 40 seconds, and a wrong postcondition ($\mathsf{sorted}(b, 0, n+1)$, $\mathsf{perm}(b, a[0 := 1])$, the counts plus one, the negated flag) leaves a goal.

7.1 Selection sort: a summary, a choice, an induction

```
b := a; i := 0;
while i < n {
  m := i; j := i + 1;
  while j < n { if b[j] < b[m] { m := j; } j := j + 1; }
  t := b[i]; b[i] := b[m]; b[m] := t;
  i := i + 1;
}
```

With

$$M_7 = \mathsf{current}(\mathsf{trace}(i,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,n_{12}) \rfloor, \lfloor \mathsf{counter}(i+1,1,n_{12}) \rfloor \\ \mathsf{do}\ m,k,j \mapsto \mathsf{case}\ b[j] < b[m] \Rightarrow j\ \ \mathsf{case}\ b[m] \le b[j] \Rightarrow m))$$

for the result of the inner loop, the goal is

$$b' = \mathsf{current}(\mathsf{trace}(a,\ \mathsf{for}\ \lfloor K \rfloor, \hat{N}, \lfloor \mathsf{counter}(0,1,n') \rfloor\ \mathsf{do}\ b,k,n_{12},i \mapsto \mathsf{swap}(b,i,M_7))) \tag{9}$$

$$\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \lfloor \mathsf{counter}(0,1,n') \rfloor, \hat{N}\ \mathsf{check}\ k,i,n_{12} \mapsto n_{12} = 0 \vee (i+1 < n \wedge \mathsf{integer}(n_{12}) + i < n)) \tag{10}$$

$$\mathsf{invs}(\mathsf{for}\ \lfloor K \rfloor, \lfloor \mathsf{counter}(0,1,n') \rfloor, \hat{N}\ \mathsf{check}\ k,i,n_{12} \mapsto n \le \mathsf{integer}(n_{12}) + i + 1) \tag{11}$$

$$\implies \mathsf{sorted}(b', 0, n) \wedge \mathsf{perm}(b', a).$$

The inner loop is a selection trace that starts at i and scans j from $i+1$; the constraints (10) and (11) are the inner test and exit, which together determine $\mathsf{integer}(n_{12}) = n - i - 1$ in every outer iteration with $i < n$. The proof needs four lemmas.

The summary of the inner loop is a plain rule on its term, true for every number of steps:

```
lemma {:rw} current_selection(b: array<int, int>, i: int, nn: Nat)
  ensures current(trace(i, for ⌊counter(0, 1, nn)⌋, ⌊counter(i + 1, 1, nn)⌋ do m, k, j ::
            case b[j] < b[m] => j case b[m] <= b[j] => m))
    == argmin(b, i, i + 1 + integer(nn))
```

It applies inside the outer update, where the inner loop is a term, and the outer trace becomes a trace of swaps of i with $\mathsf{argmin}(b, i, \mathsf{integer}(nn) + i + 1)$, with the choice among its dependencies.

The induction ranges over the counter, the choice and the loop counter, takes the two constraints on the choice as side conditions, and states the invariant, a sorted prefix below the rest, as a quantified condition over an abstract step $F(b, i, nn)$:

```
lemma {:rw} sorted_selection_steps(a: array<int, int>, n: int, m: Nat, ns: List<Nat>,
                                   ks: List<int>, is: List<int>, ns2: List<Nat>,
                                   ks3: List<int>, is3: List<int>, ns3: List<Nat>,
                                   F: (array<int, int>, int, Nat) -> array<int, int>)
  requires n == integer(m)
  requires invs(for ks, is, ns2 check k, i, nn :: n <= integer(nn) + i + 1)
  requires is == ⌊counter(0, 1, m)⌋ && ns2 == ns
  requires invs(for ks3, is3, ns3 check k, i, nn ::
             nn == zero || i + 1 < n && integer(nn) + i < n)
  requires is3 == ⌊counter(0, 1, m)⌋ && ns3 == ns
  requires forall b: array<int, int>, i: int, nn: Nat ::
    0 <= i < n && n <= integer(nn) + i + 1 && (nn == zero || i + 1 < n && integer(nn) + i < n)
      && sorted(b, 0, i) && partitioned(b, 0, i, n)
      ==> sorted(F(b, i, nn), 0, i + 1) && partitioned(F(b, i, nn), 0, i + 1, n)
  ensures sorted(current(trace(a, for ⌊counter(0, 1, m)⌋, ns, ⌊counter(0, 1, m)⌋
                               do b, k, nn, i :: F(b, i, nn))), 0, n)
    <==> true
```

The two constraints together fix $i + 1 + \mathsf{integer}(nn) = n$ for $i < n$, so every step is constrained. The quantified condition is decided by two step lemmas, that the swap with $\mathsf{argmin}(b, i, i{+}1{+}\mathsf{integer}(nn))$ extends the sorted prefix and keeps it below the rest; their side conditions are the hypotheses of the quantifier, among them the disjunction, looked up literally. The permutation follows from a lemma on any trace of steps that are permutations, over the same three dependencies, and the fact that a swap is one.

What the engine required. The constraints of the goal carry the counter of the iterations as an unused dependency, as in (10). A closed side condition is simplified before it is looked up, which drops an unused dependency and merges equal ones, so a condition written with the counter twice is not found. The conditions are therefore matched with open lists, ks, is, ns_2, which binds them to the fact's dependencies, and identified with the lists of the left-hand side by equations afterwards. A lookup modulo unused and repeated dependencies would make the open lists unnecessary.

7.2 Bubble sort: the length of each pass as a fact

Bubble sort's inner loop is a pass of adjacent swaps over $n-k-1$ positions, which a definition of the file folds into $\mathsf{bubble_pass}(b, \lfloor\mathsf{counter}(0, 1, nn)\rfloor)$. Here the engine itself has turned the constraints into one fact about the choice,

$$\mathsf{maps}(\mathsf{for}\ \lfloor K\rfloor, \hat{N}\ \mathsf{yield}\ k, nn \mapsto \mathsf{integer}(nn) + k + 1) = \lfloor\mathsf{consts}(n', n)\rfloor :$$

every pass has the length that makes $\mathsf{integer}(nn) + k + 1 = n$. The induction takes this equation as a closed condition, found among the facts, and states the invariant as a predicate indexed by the iteration, $\mathsf{bubbled}(b, k, n)$, the sorted suffix of length k above the rest, which a pass of length $n - k - 1$ extends by one; after $n' \geq n - 1$ passes it is the sorted range.

7.3 Gnome sort and the merge: the cycle as a whole

In gnome sort, the position and the array form one trace S of two accumulators, since the guard $i = 0 \vee b[i-1] \leq b[i]$ reads both and both cases change both. Its invariant, $0 \leq i$ and $b[0..i)$ sorted, holds for every run of the update, whose guards are complementary, so it is a fact about the trace, not only about the loop:

```
lemma {:rw} sorted_gnome(a: array<int, int>, ks: List<int>, n: int, x: int,
                         Sv: Trace<(int, array<int, int>)>)
  requires Sv == trace(1, a, for ks do i, b, k ::
                  case i == 0 || b[i - 1] <= b[i] => i + 1
                  case !(i == 0 || b[i - 1] <= b[i]) => i - 1,
                  case i == 0 || b[i - 1] <= b[i] => b
                  case !(i == 0 || b[i - 1] <= b[i]) => b[i := b[i - 1]][i - 1 := b[i]])
  requires x == current(Sv).0
  requires 0 <= n && n <= x
  ensures sorted(current(Sv).1, 0, n) <==> true
```

With the exit $n \leq i$, the postcondition $\mathsf{sorted}(b, 0, n)$ follows. The merge is the same shape: the pointers i and j form one trace, read by the trace of the output, and two lemmas on the atoms of the postcondition state that after at least $m + n$ steps the output is sorted on $[0, m + n)$ and its values there are the merge of the two ranges, whose counts are the sums of the counts.

What the engine required. Three forms of these lemmas failed before this one, each for a reason that applies to any lemma about a cycle:

- *No trace in a closed condition.* A condition that contains the trace itself, such as n at most the first component of the current value of $\mathsf{trace}(1, a, \ldots)$, is instantiated with fresh bound variables each time it is evaluated, so the views of that trace are recomputed and never memoized; the run did not terminate. The trace is a rule variable Sv, bound by the left-hand side to the variable of the goal, and its equation is an open condition, matched against the goal's.
- *Components read from the facts.* A closed condition $n \leq \mathsf{current}(Sv).0$ is simplified first, which replaces the read of the component by its view, and is then not found. The component is bound by an open variable, $x = \mathsf{current}(Sv).0$, matched against the fact $i = \mathsf{current}(S).0$, and the comparison is decided on x.
- *No general rule on a predicate of the postcondition.* A forward fact $\mathsf{sorted}(b, 0, i)$ about the trace together with a rule "a prefix of a sorted range is sorted" does not terminate: the rule applies to the forward fact itself. The rule is specific to the trace instead.

7.4 The distinctness check: a flag and a conjunction

The inner loop compares $a[k]$ with each later element and clears the flag d on a match. Its summary is a plain rule,

$$\mathsf{current}(\mathsf{trace}(d,\ \mathsf{for}\ \lfloor \mathsf{counter}(0,1,nn)\rfloor, \lfloor \mathsf{counter}(k+1,1,nn)\rfloor \\ \mathsf{do}\ x, i, j \mapsto \mathsf{case}\ a[k] = a[j] \Rightarrow \mathsf{false}\ \ \mathsf{case}\ a[k] \neq a[j] \Rightarrow x)) = d \wedge \mathsf{nodup}(a, k, k + 1 + \mathsf{integer}(nn)),$$

with $\mathsf{nodup}(a, k, h)$ for "$a[k]$ does not occur in $a[k+1..h)$". The outer trace is then the conjunction of nodup over the iterations, and a second lemma names it

$$\mathsf{distincts}(\mathsf{selects}(a, \lfloor \mathsf{counter}(0, 1, \mathsf{natural}(n))\rfloor)),$$

with the two constraints on the choice as conditions that fix the length of each scan to $n-k-1$. The variable of the bound in this lemma had to be named apart from the method's n: a rule variable with the name of a variable of the goal makes an open condition coincide with the fact literally, so that it holds without binding the variable.

aggregation	tests	lookup (`-teq -prove`)	MonoCera	Horn clauses for z3
$\forall$	93	61	26	0
max	4	2	2	0
min	11	8	3	0
$\sum$	17	17	10	0
total	125	88	41	0

Table 1: Proved tests of the MonoCera evaluation, with mathematical integers. Lookup: the database alone, 60 s per test, except `sum/indp5`, proved in 62 s (October 14, 2026). MonoCera, the instrumenting verifier of that evaluation, and Horn clauses solved by z3: 15 minutes per test (October 12, 2026). No result is wrong.

8 Array Programs in C: the MonoCera Tests

The MonoCera evaluation (Amilon, Esen, Gurov, Lidström, Rümmer, *Automatic Program Instrumentation for Automatic Verification*, CAV 2023) has 125 C programs derived from SV-COMP, each with a loop that fills an array and a loop that checks an assertion over it, one test per aggregation that the assertion admits ($\forall$, max, min, $\sum$). Unlike the Dafny programs of the previous sections, they carry no specification and no lemmas: the assertion is the postcondition, and everything a proof needs must come from the database. They are run with all modules of the database and mathematical integers, as in that evaluation: `-unbounded -teq` with the modules, `-prove -split -prove -split -prove`. The second and third rounds split the disjunctions that the first introduces, e.g. the case of no iteration.

Two things distinguish the C programs. A `for` loop with a body of conditionals gives one case per path, also where the paths agree, e.g. the counter i with two cases $i + 1$, which the database merges first (trace_merge_cases_ne). And the harness branches, `if (N <= 0) return 1`, so the goal has one branch per path, of which one carries the loops. The iteration constraint of a loop with guarded cases also states that one of the guards held, e.g. $i \bmod 2 = 0 \vee \neg(i \bmod 2 = 0)$: the completeness of the guards in the context of the update (Section 3.6) is a fact of the goal.

8.1 Loops in sequence: sums of filled arrays

```
for (i = 0; i < N; i++) { if (i % 2 == 0) a[i] = 2; else a[i] = 0; }   // brs2
for (i = 0; i < N; i++) { if (i == 0) sum[0] = 0; else sum[0] = sum[0] + a[i]; }
__VERIFIER_assert(sum[0] <= 2 * N);
```

The second loop reads the final array of the first, and the sum is kept in the cell 0 of a one-element array, so its trace is a trace of stores at the constant key 0. The proof needs one lemma per shape, each landing in the normal form that the next one expects:

1. *The counters.* The traces of i are counters, the constraints and exits give $\mathsf{integer}(n) = N$ for both loops, and inside each iteration i is the iteration k.
2. *The first loop is stores of a map.* Its update stores a constant chosen by a condition on the index:

$$\begin{aligned}&\mathsf{stores_index_cases_def}:\\ &\quad \mathsf{trace}(a_0,\ \mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor, \lfloor\mathsf{counter}(0,1,n)\rfloor\\ &\qquad \mathsf{do}\ a,k,i \mapsto \mathsf{case}\ P(k) \Rightarrow a[k := c]\ \ \mathsf{case}\ \neg P(k) \Rightarrow a[k := d])\\ &\quad = \mathsf{stores}(a_0, \lfloor\mathsf{counter}(0,1,n)\rfloor, \mathsf{maps}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k \mapsto \mathsf{if}\ P(k)\ \mathsf{then}\ c\ \mathsf{else}\ d))\end{aligned}$$

   with P the parity, abstracted from the guard.
3. *The read in the second loop.* $a'[k]$ is the stored value at k (current_stores, with the membership and position of k in the counter, occurs_counter_up and indexof_counter_up), decided in the context of the iteration, where $0 \le k < \mathsf{integer}(n)$.

4. *The cell is a sum.* The trace of stores into the cell is read as the scalar sum:

$$\begin{aligned}&\mathsf{select_current_trace_reset_add}:\\&\quad\mathsf{current}(\mathsf{trace}(a_0,\dots\ \mathsf{case}\ k=0\Rightarrow a[j:=c]\ \ \mathsf{case}\ 0<k\Rightarrow a[j:=a[j]+G(k)]))[j]\\&\quad=\mathsf{if}\ n=0\ \mathsf{then}\ a_0[j]\ \mathsf{else}\ c+\mathsf{sum}(\mathsf{maps}(\mathsf{for}\ \lfloor\mathsf{counter}(0,1,n)\rfloor\ \mathsf{yield}\ k\mapsto\mathsf{if}\ 0<k\ \mathsf{then}\ G(k)\ \mathsf{else}\ 0))\end{aligned}$$

5. *The bound.* sum_cases_below bounds a sum of terms that are each 0, c or d by e; its left-hand side is the comparison $\mathsf{sum}(\dots)\le e$, whose right side binds e, so the bound need not be a fact.

The test `sum/ms2`, which stores i mod 2, differs only in the bound (sum_mod_below), and `sum/conda` only in the first loops: an array filled with ones and then updated in place by cases on the old value,

$$\begin{aligned}&\mathsf{stores_cases_def}:\\&\quad\mathsf{trace}(a_0,\ \mathsf{for}\ \dots\ \mathsf{do}\ a,k,i\mapsto\mathsf{case}\ P(a[k])\Rightarrow a[k:=c]\ \ \mathsf{case}\ \neg P(a[k])\Rightarrow a[k:=G(a[k])])\\&\quad=\mathsf{stores}(a_0,\dots,\mathsf{maps}(\mathsf{for}\ \dots\ \mathsf{yield}\ k\mapsto\mathsf{if}\ P(a_0[k])\ \mathsf{then}\ c\ \mathsf{else}\ G(a_0[k]))),\end{aligned}$$

since each position is read only in its own iteration, before it is written. Its first case stores a constant, not $G'(a[k])$: by the time the lemma is tried, the read $a[k]$ inside the update has already become 1 (the stores of the previous loop, read in the context of the iteration), so the program's $a[k]+1$ is 2. A higher-order instance must contain the term it abstracts, and the constant 2 contains no $a[k]$.

What this needs. Loops in sequence compose through their current values: each loop reads the previous array, the read is resolved in the context of the iteration, and only then is the shape of the next loop recognized. The lemmas are written for the goal as the engine has simplified it, not for the program text. And where the data has a different shape, a cell of an array instead of a scalar, the lemma for the new shape lands in the normal form of the old one (the right-hand side of the scalar sum), so that everything after it is shared. One detail of the engine is needed here: the instance $(z)\mapsto a[z]$ of a higher-order variable must not be eta-reduced to the array a, since $a(k)$ applied as a function is not the read $a[k]$ that the rules are about; eta reduction applies only to functions of array type.

8.2 Strided writes: a quotient in the bound (open)

```
if (CELLCOUNT % 2 != 0) return 1;   // pr2
for (i = 1; i <= CELLCOUNT / 2; i++) {
  volArray[i*2 - 2] = (3 >= MINVAL) ? 3 : 0;
  volArray[i*2 - 1] = (1 >= MINVAL) ? 1 : 0;
}
for (i = 0; i < CELLCOUNT; i++)
  __VERIFIER_assert(volArray[i] >= MINVAL || volArray[i] == 0);
```

The loop stores pairs at $2k$ and $2k+1$, with values chosen by three cases on `MINVAL` that do not depend on the iteration. The module for strided stores reads a cell of the result:

$$\begin{aligned}&\mathsf{select_current_trace_stride2_cases}:\quad R\Leftrightarrow(\neg P\wedge\neg Q)\Longrightarrow\\&\quad\mathsf{current}(\mathsf{trace}(a_0,\dots\ \mathsf{case}\ P\Rightarrow a[2k:=X_1][2k+1:=Y_1]\ \ \mathsf{case}\ Q\Rightarrow\dots\ \ \mathsf{case}\ R\Rightarrow\dots))[j]\\&\quad=\mathsf{if}\ 0\le j<2\,\mathsf{integer}(n)\ \mathsf{then}\ \dots\ \mathsf{else}\ a_0[j],\end{aligned}$$

by the parity of j and the guards. Its side condition requires the guards to be exhaustive; the lemma fixes two stores and three cases on its left-hand side. It fires on the reads of the checking loop, and every value it produces satisfies the assertion. The goal that remains is the range above the stores:

$$2\,(C/2)<C\ \wedge\ C\bmod 2=0\ \wedge\ 2\,(C/2)=i\Longrightarrow\dots a[i]\dots$$

for C the size: the loop ran $C/2$ times, the read at $i=2\,(C/2)$ lies beyond the stores, and the case is impossible since C is even. The arithmetic knows the bounds of $C/2$, but not that $C \bmod 2$ is $C-2\,(C/2)$, so it does not see the contradiction.

What this needs. Remainders as linear terms over quotients: with $t \bmod k = t - k \cdot (t/k)$ for a positive constant k, the facts $C \bmod 2 = 0$ and $2\,(C/2) < C$ are inconsistent, and the test closes. A lemma stating the fact for one form of the equation, e.g. $x \bmod 2 = 0 \Rightarrow (n = x/2 \Leftrightarrow x = 2n)$, depends on the form in which the engine keeps the equation, which changes with every improvement of the arithmetic; the remainder belongs to the decision procedure. The tests with s per-cell guards, `pr3` to `pr5`, need 2^s combinations of cases, which a lemma with a fixed number of cases does not provide: a lemma schema with the number of stores and cases as a parameter.

8.3 Stores ahead, in both directions

```
for (i = 0; i < SIZE; i++) { if (i + 1 < SIZE) a[i+1] = i; a[i] = i; }       // rewmif
for (i = SIZE - 1; i >= 0; i--) { if (i - 1 >= 0) a[i-1] = i - 2; a[i] = i; } // rewmifrev
for (i = 0; i < SIZE; i++) __VERIFIER_assert(a[i] >= i);
```

Each iteration writes one cell ahead before it writes its own; the value written ahead is overwritten by the next iteration, so a cell holds what its own iteration wrote last, except the cell past the last iteration, which holds the value written ahead. The module for strided stores has one lemma per direction; for the loop down from s,

```
lemma {:rw} {:teq} select_current_trace_ahead_down(a0: array<int, int>, n: Nat, s: int,
                                                   F: (int) -> int, G: (int) -> int, j: int)
  ensures
    current(trace(a0, for ⌊counter(0, 1, n)⌋, ⌊counter(s, -1, n)⌋ do a, k, i ::
              case 1 <= i => a[i + (-1) := F(i)][i := G(i)]
              case i < 1 => a[i := G(i)]))[j]
      == if s - integer(n) < j && j <= s then G(j)
         else if j == s - integer(n) && 0 <= j && 0 < integer(n) then F(j + 1)
         else a0[j]
```

It fires on the reads of the checking loop with $s := \texttt{SIZE} - 1$, $F := (z) \mapsto z - 2$ and $G := (z) \mapsto z$, and the assertion holds in every branch; the test closes under $\forall$ and min.

What this needs. The pattern contains two literals that a matcher must treat modulo the normal form of the goal: the step -1 of the counter dependency, matched through the definition of the program's counter, and the store position $i+(-1)$, which the goal has as $i-1$, a sum with a negative literal. The engine compares a negative literal among the summands as the negation of its absolute value, so the lemma can be written as the program reads. Without that, each such position needs a higher-order variable $P(i)$ pinned to $i - 1$ by a side condition, and each literal step a variable d with $d = -1$. The third variant of the test, `rewnifrev2`, writes ahead upwards in a loop that runs downwards, a shape for which there is no lemma.

8.4 Recurrences on the previous cell

```
for (i = 0; i < N; i++) { if (i == 0) a[0] = 4; else a[i] = a[i-1] + 4; }   // ifeqn2
for (i = 0; i < N; i++) { if (i == 0) b[0] = 1; else b[i] = b[i-1] + a[i-1]; }
for (i = 0; i < N; i++) __VERIFIER_assert(b[i] == 2*i*i + 2*i + 1);
```

A store at k of a value read at $k - 1$ is not a store of a map: the value depends on the trace, not on the initial array. The aggregation module names the recurrence by its closed form,

```
lemma {:rw} {:teq} select_current_trace_recurrence(a0: array<int, int>, n: Nat, c: int, j: int,
                                                   G: (int) -> int)
  ensures current(trace(a0, for ⌊counter(0, 1, n)⌋ do a, i ::
            case i == 0 => a[0 := c] case 0 < i => a[i := a[i - 1] + G(i)]))[j]
    == if 0 <= j && j < integer(n)
       then c + sum(maps(for ⌊counter(1, 1, natural(j))⌋ yield k :: G(k)))
       else a0[j]
```

The first loop gives $a'[j] = 4 + \mathsf{sum}(\mathsf{maps}(\mathsf{for}\ \lfloor \mathsf{counter}(1, 1, \mathsf{natural}(j)) \rfloor\ \mathsf{yield}\ k \mapsto 4)) = 4 + 4j$ (the map of a constant and its sum). In the second loop the summand $G(k) = a'[k - 1]$ is then $4k$,

so $b'[j] = 1 + \mathsf{sum}(\mathsf{maps}(\mathsf{for}\ \lfloor \mathsf{counter}(1, 1, \mathsf{natural}(j)) \rfloor\ \mathsf{yield}\ k \mapsto 4k))$; the map of a multiple of the counter is a counter, $\lfloor \mathsf{counter}(4, 4, \mathsf{natural}(j)) \rfloor$ (maps_counter_scale), whose sum is the arithmetic series (sum_counter), $2j^2+2j$. The read of the checking loop at its index meets the assertion exactly.

What this needs. The read $a[i-1]$ of the pattern must match the goal's $a[k+(-1)]$, the same negative literal as above; the higher-order summand G must take the read of the other array as a whole; and the closed forms of maps and sums over counters must reach polynomials, which the arithmetic compares. The recurrences with other increments, products or conditions (`ifeqn3` to `ifeqn5`, `sqm`, `condm`) need their own closed forms; a recurrence solver as a generator of such lemmas would cover the polynomial ones.

9 The Open Algorithms: What They Need

Twenty of the thirty programs of the folder of algorithms are proved. Of the other ten, eight leave a goal and two stop because the rewriting exceeds its budget. Their lemmas, written at the beginning of each file, mostly describe inner loops as families of fresh lists, one per inner trace, with the facts of the inner loop distributed over the outer iteration constraint; they do not match the closed form of Section 2. For each, this section states what a proof needs, and, where an attempt was made, what stood in its way.

9.1 Insertion sort: an inner loop of data-dependent length

Insertion sort moves the new element down while it is less than its predecessor: `while 0 < j && b[j] < b[j - 1] { swap; j := j - 1; }`. Unlike the inner loops of Section 7, the number of inner iterations is not determined by the bounds: the constraints over the inner history state that the comparison held in every inner step, and the exit that it failed or j reached 0. The closed form has the inner loop as

$$\mathsf{current}(\mathsf{trace}(b,\ \mathsf{for}\ \lfloor \mathsf{counter}(0, 1, nn) \rfloor, \lfloor \mathsf{counter}(i, -1, nn) \rfloor \\ \mathsf{do}\ x, k', j \mapsto x[j := x[k - k']][k - k' := x[j]]))$$

with the position $j-1$ expressed through the outer iteration k, and the constraints are nested inside the outer iteration constraint, over the history of the outer trace, which the closed form spells out as a term.

What it needs. A step lemma whose conditions are the inner constraint and exit, and whose conclusion is that the result extends the sorted prefix by one, used by an induction over the outer trace with the choice among its dependencies. Both lemmas have conditions that contain inner and outer traces as terms, which runs into the cost of views recomputed for each instance (Section 7.3); a summary that names the inner loop by an abstract $\mathsf{insert}(b, i, nn)$ first, as a plain rule, and states the comparisons as facts about the named result, would keep the traces out of the conditions.

9.2 The partitions: cycles of three accumulators

In the partition around a pivot, the scanning index j joins the cycle of the array and the boundary i, although j is a counter: the guard reads $b[j]$, and every result of a guarded body reads the variables of its guard, including j's two cases $j+1$, which are equal. The invariants hold for every run of the update ($0 \le i \le j$, the elements below i less than the pivot, those from i below j at least the pivot, a permutation), so lemmas in the style of gnome sort are sound. But the program alone takes 34 s, and 102 s without the aggregation module, since the views of a cycle of three accumulators are expensive, and with the lemmas the rewriting exceeds its budget.

What it needs. A read relation that merges equal cases first, or takes the guards only for the results that differ between the cases, would keep the counter outside the cycle, where the database recognizes it, and leave a cycle of two. The three-way partition has the same structure with two boundaries.

9.3 The first duplicate: an early exit by a flag

The first duplicate scans the pairs $i < j$ like the distinctness check, but stops both loops when the flag is set. The inner flag is the one of Section 7.4, $r \vee \neg\mathsf{nodup}(a, i, i + 1 + \mathsf{integer}(nn))$. The outer loop runs while the flag is false, so its constraint states that the flag was false at the head of every outer iteration, over the history of the outer trace; the exit of the outer loop is a disjunction, $n \leq i \vee r$, and the exit of the inner loop depends on the flag as well.

What it needs. Two rules on the atom $\mathsf{distincts}(\ldots)$, one per case of the outer exit: if the flag is set, the last iteration found a duplicate within $[0, n)$; if it is not, every iteration ran with its full length and found none. Both take the constraint on the flag at the heads as a condition, which contains the outer trace as a term; with the postcondition as two implications, each goal has its literal of the exit.

9.4 The reversal: a fact lost in a normal form

The swaps of k and $n - 1 - k$ are recognized as a reversal by a lemma of the file, whose conditions are linear: $2n' \leq c + 1$ and $c \leq 2n'$ for the start $c = n - 1$ of the counter down. They follow from the facts if $\mathsf{integer}(n') = i$ for the final value i of the counter. The database rewrites $\mathsf{integer}(n) = x$ into $n = \mathsf{natural}(x)$ under the condition $0 \leq x$, and the result no longer states $0 \leq x$, which is lost if it followed from the equation itself. The equivalence $\mathsf{integer}(n) = x \Leftrightarrow 0 \leq x \wedge n = \mathsf{natural}(x)$ holds without a condition and keeps the fact; with it, the conditions of the reversal follow.

9.5 Witnesses, duplicates and the step budget

- *An existential postcondition* (`max_element`): the maximum is

  $$m = \mathsf{current}(\mathsf{maxs}(a[0], \mathsf{selects}(a, \lfloor \mathsf{counter}(1, 1, n) \rfloor))),$$

  and $\exists k.\ a[k] = m$ needs a witness. A lemma whose left-hand side is the existential does not apply: the matcher has no case for quantifiers. The same program with the postcondition $\mathsf{occurs}(m, \ldots)$ in the vocabulary of the database closes; quantified left-hand sides, or witnesses taken from the terms of the goal, would close the other.
- *The transpose* of the folder keeps two folds and lemmas for fresh lists; the same program with one lemma on the nest (Section 5.3) closes.
- *The step budget*: the cocktail sort, with two inner loops per outer iteration, and the counting sort, with four loops in sequence and a scatter through a table of positions, stop because the rewriting exceeds its budget, with a rule on counters or sums as the last step. The last step of a rewriting that does not terminate is rarely its cause: finding the cycle needs the rules of the file in pairs on the program, or a report of the rule applications that repeat.

10 Principles

This section collects what the examples have in common: the shape that a proof by lookup takes, how to write a lemma that the engine applies, how to keep such lemmas sound and the rewriting terminating, and how to find out why a lemma does not apply.

10.1 The shape of a proof

Every proof of this document is built from lemmas of the following kinds, from generic to specific. The first three are the database's; the others are knowledge about one algorithm, stated at the beginning of its file.

1. **The database resolves single loops.** Counters, sums, stores and their reads, selects, counts, minima and maxima are introduced and accelerated without anything written for the program (Sections 4, 5 and 8.1).
2. **Inner loops reuse the lemmas of single loops.** An inner loop is a value of the outer update, so the same lemmas apply to it where it occurs, with the facts of the outer iteration in the context. Only the numbers of inner iterations are fresh; once the constraints determine them, they are a constant list, and the inner traces have a known number of steps (Section 5).
3. **Constraints become named facts.** An iteration constraint that a later step needs at a position is replaced by an atom that names it, $\neg\mathsf{occurs}(x, \mathsf{selects}(a, \ldots))$, and a second lemma uses the atom at any position (Section 4).
4. **A loop that is only meaningful as a whole gets a name.** A definition of the whole loop with a total update is folded where its body occurs, and forward facts about the named loop give the bounds that the postcondition needs (Section 6).
5. **A summary states what a loop nest computes**, as a rule whose left-hand side is the term of the goal: the nest of the transpose (Section 5.3), and for the sorts the inner loop as an abstract step, a plain rule on the inner loop's term, e.g. argmin (Section 7).
6. **An induction lemma states the invariant of an outer loop** as a quantified side condition over an abstract step, decided by step lemmas on atoms. No invariant is inferred; its shape is the lemma's knowledge, and the engine proves the step.
7. **Rules on the atoms of the postcondition** characterize a named result: the comparisons with the partition points of a sorted array, the regions of a partition, the cases of an exit. The postcondition is never rewritten as a whole; each atom finds its rule.
8. **Name the normal form.** Postconditions are stated in the vocabulary of the database where the standard form has no witness or no matchable shape (occurs rather than $\exists$, count over selects rather than a quantified multiset equation), and quantified preconditions as atoms of the specification. A lemma for a new shape of data lands in the normal form of the old one, so that the rest of the proof is shared (Section 8.1).

A data-dependent number of iterations is no obstacle to this shape, as long as the property can be stated on the final values and the exit is among the facts: the binary searches have it, and so do the merge and gnome sort.

10.2 Writing a lemma that the engine applies

1. **Write for the goal, not for the program.** The engine simplifies before it matches: reads are resolved, guards decided, cases merged. A lemma is written for the goal as the engine prints it: the first case of stores_cases_def stores a constant, because the read in the update was resolved before the lemma was tried (Section 8.1); the transpose's lemma has $\mathsf{natural}(n)$ as the number of inner iterations, because that is what the constraints determine.
2. **The left-hand side is a term of the goal.** A variable of the goal matches through its definitions (Section 3.1), so a lemma on $\mathsf{current}(\mathsf{trace}(\ldots))$ applies to b with $b = \mathsf{current}(B)$, inside $\mathsf{sorted}(b, 0, n)$ and elsewhere. A left-hand side that the goal does not contain, e.g. the trace of an inner loop described by separate lists, never applies, however true it is.
3. **Higher-order variables abstract a term of the match.** $G(k)$ matches any term over the bound variable, by abstraction; its instance must contain the term it abstracts. Arithmetic on a bound variable, $a[i-1]$, is matched modulo the normal form of sums, including negative literals; arithmetic on two rule variables, $m+n$, is not a pattern, and is written as a variable s with $s = m+n$ as a side condition.
4. **Side conditions are matched while open, looked up when closed.** A condition with

an open variable binds it from a fact; a closed one is first looked up among the facts modulo the classes and canonical forms, and only then decided. An iteration constraint can therefore be a closed condition, written as the goal states it, unless the goal's constraint has an unused or repeated dependency: the instance is simplified before the lookup, which drops such dependencies, and is then not found; such a condition is matched with open lists, identified with the lists of the left-hand side by equations (Section 7.1). A quantified hypothesis is never matched; the specification provides an atom for it.

5. **Guards in the context of the update.** A pattern's guard matches a program's guard if its conjuncts follow from it in the context of the update, where the iteration constraint holds. A lemma may therefore guard its cases by what the loop guarantees ($lo < hi$), and may add a case for the complement, which the program's cases never need to cover (Section 3.6).
6. **Traces as rule variables.** A lemma about a trace of the goal names it by a rule variable, bound by the left-hand side to the goal's variable, with the equation of the trace as an open condition; the components are read by open variables, $x = \mathsf{current}(Sv).0$, matched against the facts. A condition that contains a trace as a term has its views recomputed at every evaluation, and a closed read of a component is replaced by its view before it is looked up (Section 7.3).
7. **Dependencies.** A pattern may iterate over more dependencies than the trace, e.g. the counter and a loop variable equal to it; repeated dependencies of the pattern take the same dependency of the goal. Further dependencies that the update does not read, hs, are selected by no variable of the pattern and left aside.
8. **Variables named apart.** The variables of a lemma are not renamed apart from those of the goal: a method variable with the name of a rule variable can make an open condition coincide with a fact literally, so that it holds without binding the variable. Methods and lemmas of a program name their variables apart from those of the database (x, n, Ks, ...).

10.3 Soundness

The lemmas of the database are proved in Lean; the lemmas of a program are axioms, and the abstract predicates (sorted and perm are defined in the database, but argmin, insert, bubbled and the other predicates of the sorts are not) are axiomatized by them. The discipline below replaces the missing check.

- **A fact must hold for every value of its terms.** A forward fact about a named trace is assumed wherever the term occurs, not only where a loop produced it. The bounds of the bisection hold for the loops of the programs but would be false for an unspecified step after the exit; the definition is made total by a case that keeps the state, so that the fact holds for every run (Section 6). The same applies to any lemma with arbitrary guards: a last case for their complement, with a result that keeps the conclusion true, makes complete guards unnecessary as a premise.
- **An induction constrains every step.** A quantified step condition over $0 \le k < n$ with $n \le \mathsf{integer}(m)$ steps leaves the further steps unconstrained. The number of steps is fixed exactly, or the invariant holds for the steps beyond the last meaningful one, or the result holds for any number of iterations beyond a bound.
- **A condition must not be provable by the conclusion.** A side condition is matched in the context of the facts, where other rules fire; a rule that rewrites the condition using the atom that the lemma is about derives the atom from itself. The safe form of a condition is an atom matched literally, or a fact of the loop.
- **Every result is checked on a wrong postcondition.** A lemma set that also closes $\mathsf{sorted}(b, 0, n+1)$, the sum plus one, or a negated flag is unsound or vacuous. For the C programs, a false assertion leaves a residual that shows what the lemmas computed, e.g. the sum $N-1$ against a bound $N-2$.
- **A killed run prints no goal.** A run that exceeds its time limit prints nothing and looks like a proof; every result is read with its exit status, and long runs use a private copy of the

tool, since a rebuild during a run crashes it.

10.4 Termination

Rewriting cycles by one mechanism. An iteration constraint as a side condition with open variables is matched against the iteration constraints of the goal, which builds the context of each fact's combinator and rewrites the reads of its dependencies there; if the left-hand side of the rule matches one of those rewritten reads, the rule recurs through the same fact one level deeper, until the step budget is exhausted. Both ingredients are needed: a condition over lists that the left-hand side binds, and a left-hand side general enough to match inside that context, such as a bare read, a comparison of a bound variable, or an equation. A rule with such conditions therefore has a left-hand side that no rewriting of a read can produce: an iteration constraint with a specific body, or an atom of the specification. The order of the modules also decides which abstraction a fact gets, since rules are tried in the order of the files. Where the budget is exceeded, the reported rule is the last step of the cycle, not its cause.

10.5 Diagnostics

- With `-rules`, a proof reports the lemmas it applied, per goal, including the instances of forward facts; a residual goal points to the missing lemma.
- `-debug match` reports the rules on traces whose left-hand side fails, with the case of the program that no case of the pattern covers and the guard that fails; `-debug conditions` the closed side conditions that fail, with the facts on their variables; `-debug forward` the instances of forward facts; `-debug rewrite` every step, including the views of cycles.
- *Probes*: a copy of a lemma with its first k conditions and a fresh atom as right-hand side shows in the goal the largest k whose conditions hold. *Pair runs*: the rules of a file in pairs on a program that does not terminate find the pair that cycles.
- *Control runs*: a lemma that is believed to fire is removed or broken, and the goal must stay open; a definition that is believed to fold is made a plain declaration, and the fold must stop.

10.6 The same principles on C programs

The MonoCera tests show the principles without hand-written specifications.

- **The shape of the data decides the lemma, not the idea.** A sum kept in a cell of an array is a trace of stores; the lemma for it has the right-hand side of the scalar sum, so the database continues from there.
- **Left-hand sides bind, side conditions must be facts.** sum_cases_below has the bound in its left-hand side, a comparison, so the bound need not be among the facts.
- **Specific lemmas are summaries.** The strided stores and the stores ahead fix the number of stores and of cases on their left-hand sides, as the summary of an algorithm fixes its loop nest; strides with per-cell cases want a schema with the number of cases as a parameter.
- **The checking loop is an instantiation at a witness.** The assertion of the checking loop becomes an iteration constraint over its earlier iterations and a goal at the current index; the step from the constraint of one loop to a position of another is the one of the linear search, an atom and its instantiation, and needs the closed form of the filled array.
- **Lookup against inference.** Horn clauses for z3 prove none of the 125 tests: the quantified invariants over arrays, $b[k] = 2k^2 + 2k + 1$ for all $k < i$, are not found. A lemma found once serves every loop of its shape, and the price is that a loop without a lemma stays open.

11 Insights

11.1 The method is lookup; the difficulty is the engine

Verification by lookup states, once, what loops of a shape compute, and finds the loops of a program among the shapes. On the MonoCera tests this proves within seconds what inference of quantified array invariants does not find in fifteen minutes, and the price is that a loop without a lemma stays open. The examples of this document strained not the method but the engine that performs the lookup. Each practice that a lemma writer would otherwise need is a symptom of a missing capability of the engine, and the engine of Section 3 is organized by the symptoms it removes:

capability	what a lemma would need without it
congruence closure as the context	open variables for every trace of the goal, tied by equations; the whole atom of the postcondition as left-hand side, since a variable argument is not expanded; equal counters kept apart
closed conditions looked up	every iteration constraint as a condition restated with fresh lists, identified with the goal's by equations
a decision procedure for linear arithmetic	lemmas for the bounds of a midpoint; conditions stated in the exact form of the goal's facts, since only difference constraints are combined
canonical forms of literals	a higher-order variable pinned to $i - 1$ for every store position, a variable $d = -1$ for every downward step
views of cycles	per-variable equations, each recursive through the others, and every lemma tying the dependency lists back to their traces
patterns over whole cycles, with the constraint in the update	a lemma per pointer, unable to state that the pointers move together
forward facts with triggers	rules on atoms for facts that no atom mentions, and abstractions of constraints that hide them from later rules
suspension of the rule being applied	no condition may mention the left-hand side, so the exit of a loop cannot be stated about its own trace

What remains for the lemma writer is the algorithm's knowledge: what a loop nest computes, which invariant an outer loop keeps, and the vocabulary in which the postcondition is stated.

11.2 The representation: equations per component

A loop over variables $\bar{x}$ is, semantically, one fold over its iterations with the state tuple as the accumulator. A system of one equation per variable is this fold decomposed by projection: each equation steps one variable, with the others among its dependencies. Where the variables form a layering (a counter, an index over it, an array written at the index, a sum over the array), the decomposition is the compositional reasoning of the method: each equation meets a small lemma of the database and the result is substituted downstream, which is why single loops and the arrays of SV-COMP need nothing written for the program. A single fold over the whole state would first have to rediscover that layering before any lemma on a counter or a sum could apply.

Where the variables read each other, the decomposition is the obstacle: the pointers of a merge, the bounds of a binary search, the position and the array of gnome sort give equations that are individually meaningless, each recursive through the others. The partition by strongly connected components takes the best of both: a variable in no cycle keeps its own equation, where the database's lemmas apply, and a cycle becomes one trace of several accumulators, which a lemma may name as a whole. The per-variable view of a component is not lost; it is computed on demand, as a projection, when a lemma about a single accumulator needs it. The lemmas keep their form, and the engine translates between the representations.

The same holds for nested loops. An inner loop described by fresh lists of the outer loop, one per inner trace and its final value, puts the facts that belong together into different combinators, and every substitution between them is a lemma. As a value of the outer update, the inner loop is one term, the lemmas of single loops apply to it, and only the number of its iterations is a choice of the outer loop, to be determined by its exit.

11.3 Soundness by totality

A lemma that holds for the loops of programs is not yet a lemma: it is applied wherever its term occurs, also to traces that no loop produced, e.g. a trace past its exit, whose steps are unspecified. The bisection shows the failure and the remedy. Stated for a partial update, its bounds derive false; stated for a total definition, which keeps the state where the program's step is undefined, they hold for every run and have a proof. The engine supports this by matching case lists in one direction: the program's guards are complete in the context of its updates, by construction, so a lemma may be total where the program is partial. This turns a premise that every lemma with guards would carry, and that no engine checks, into a case of the lemma's definition.

11.4 What the open examples need

- *Closed conditions looked up modulo unused and repeated dependencies*: the constraints of nested loops carry the counter of the iterations unused, so a summary of a sort must match them with open lists and identify these by equations, which is the bookkeeping that the lookup is meant to remove.
- *Views memoized for terms, not only for variables*: a condition that contains a trace is instantiated with fresh bound variables at every evaluation, and its views are recomputed; lemmas about cycles must name the trace by a rule variable and read its components through open variables, and the insertion sorts and the first duplicate, whose constraints contain traces as terms, are blocked by it.
- *Normal forms that keep facts*: $\mathsf{integer}(n) = x$ becomes $n = \mathsf{natural}(x)$ only under $0 \leq x$, which is then lost; the unconditional form $0 \leq x \wedge n = \mathsf{natural}(x)$ keeps it (the reversal).
- *Remainders as linear terms over quotients* in the arithmetic, which closes the strided writes of Section 8.2 without a lemma for one form of an equation.
- *A more precise read relation*, taking the guards only for the results that differ between the cases, so that a counter of a guarded body stays outside the cycle of the variables that the guard reads; the partitions have a cycle of three accumulators for that reason, whose views are too expensive.
- *Lemma schemas*, with the number of stores, cases or dependencies as a parameter, in place of one lemma per arity.
- *Quantified left-hand sides or witnesses* for existential postconditions: the matcher has no case for quantifiers.
- *Diagnostics for non-termination*, which report the rule applications that repeat rather than the last one.
- *Proofs for the lemmas of programs*: definitions of the abstract predicates and functions of the sorts in Lean, as the database's lemmas have them, so that the summaries and step lemmas are checked rather than trusted.

AI Declaration

The examples of `examples/dafny/algorithms` and `examples/dafny/state`, their specifications and lemmas, the lemmas for the MonoCera tests, the reruns reported here and this document were produced by coding agents (Claude Code with Claude Opus) between October 9 and 14, 2026, under the direction of the author, who chose the problems, set the rules of the work (no unfolding of trace combinators in lemmas for programs, every result checked on a wrong postcondition, runs read with their exit status), decided the open questions, and reviewed the results. The lemmas of the programs are axioms that no proof checks (Section 10.3), and the text should be read as the agents' account of the experiments, reviewed by the author.